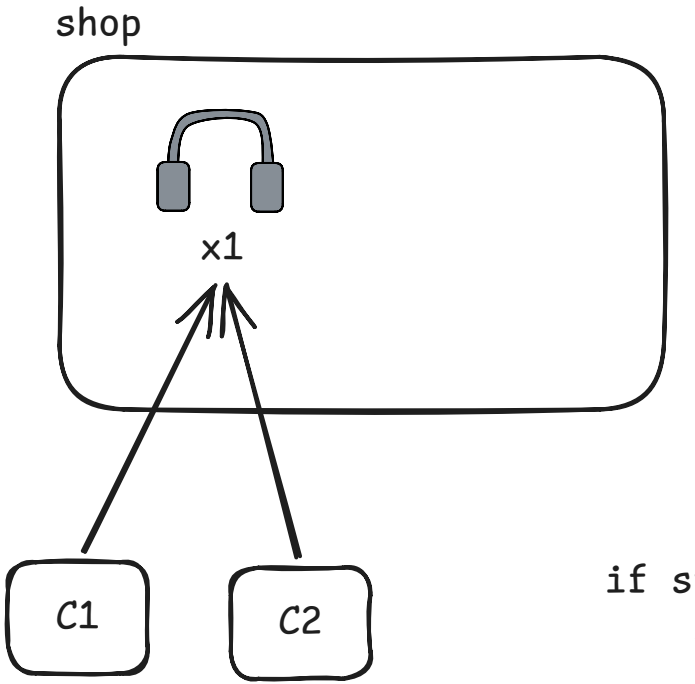
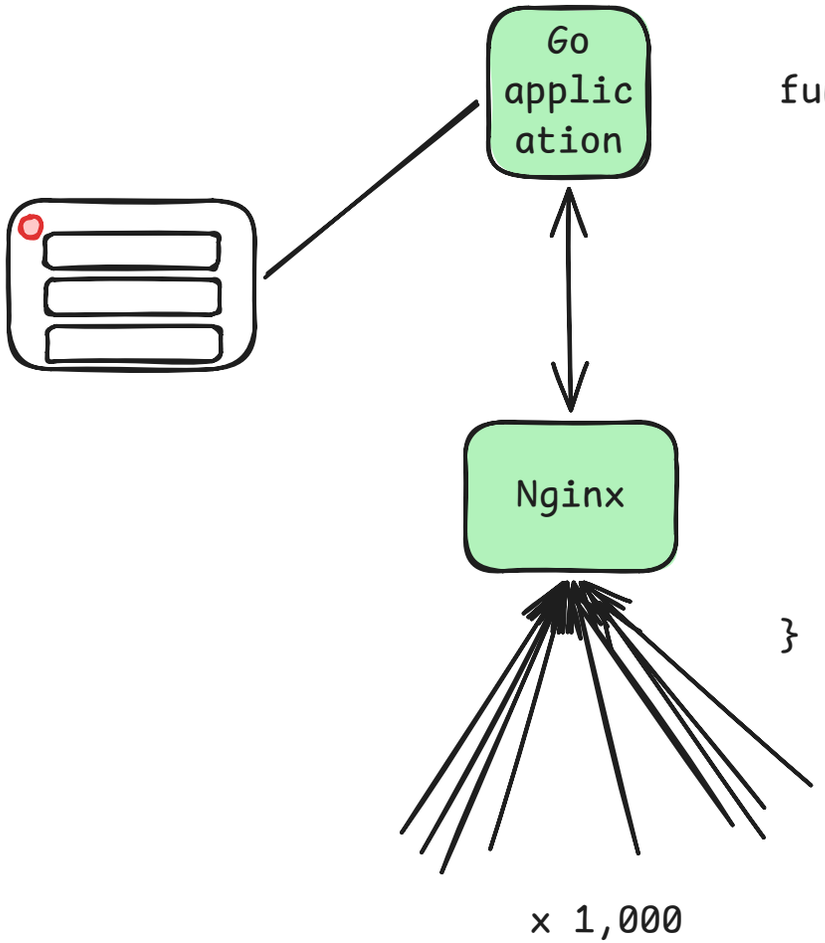
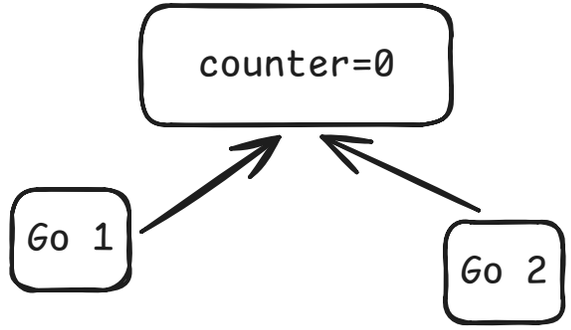
Lecture 10 - Synchronization




```
if stock > 0 {
    stock--
    fmt.Println("success!")
}
```




```
func main() {
    var counter int

    // 1,000 visitors
    for i:=0; i < 1000; i++ {
        go func() {
            counter++
        }()
    }

    fmt.Println("Counter: ", counter) // 572
}
```




```
Go 1: counter = 0
Go 2: counter = 0
Go 1: counter = counter + 1
Go 2: counter = counter + 1 // 0 + 1
Go 1: counter = 1
Go 2: counter = 1
```

sync

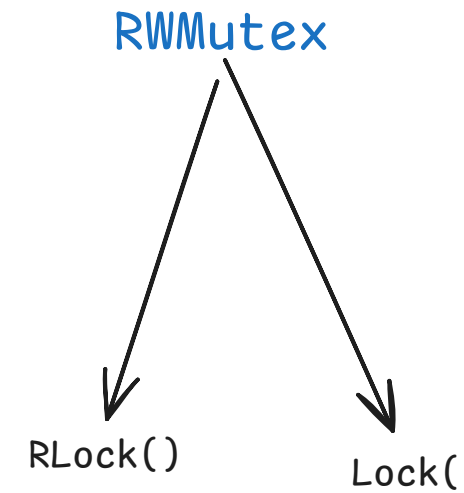
WaitGroup

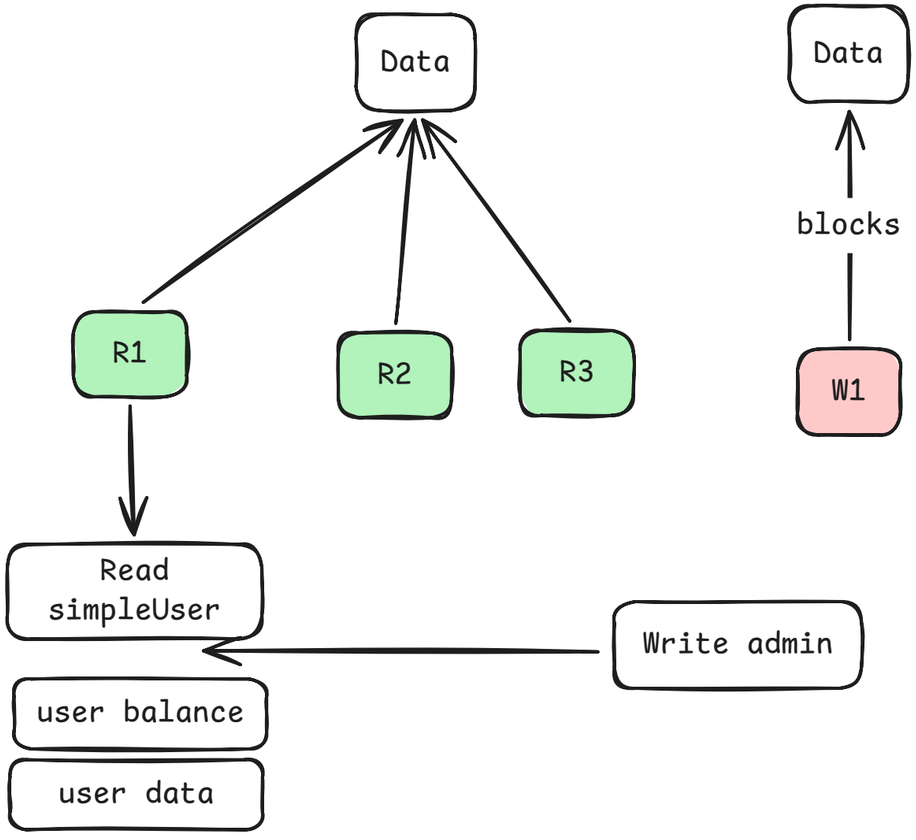
```
func main() {
    var counter int
    var wg sync.WaitGroup // 1,000 tasks

    // 1,000 visitors
    for i:=0; i < 1000; i++ {
        wg.Add(1)

        go func() {
            defer Done()
            counter++
        }()
    }

    wg.Wait()

    fmt.Println("Counter: ", counter) // 990
}
```

```
Add()
Done()
Wait()
```

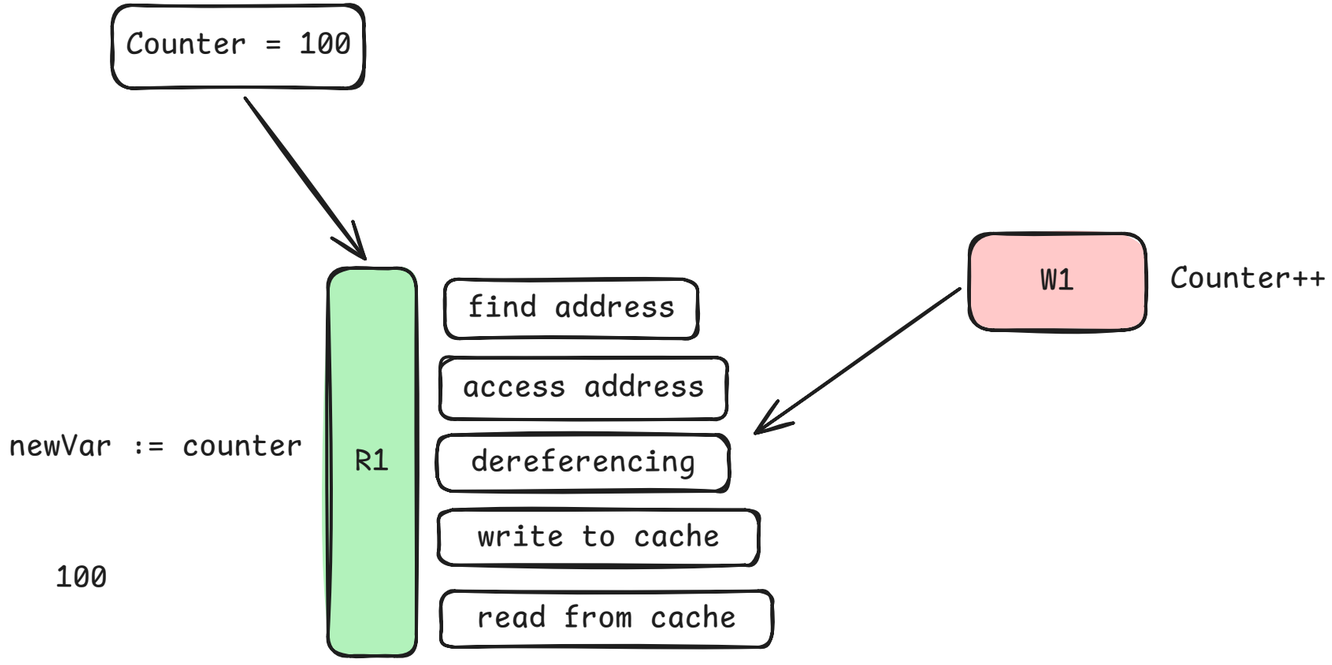
Mutex

```
func main() {
    var counter int
    var mu sync.Mutex
    var wg sync.WaitGroup // 1,000 tasks

    // 1,000 visitors
    for i:=0; i < 1000; i++ {
        wg.Add(1)

        go func() {
            mu.Lock()
            defer mu.Unlock()
            defer Done()
            counter++
        }()
    }

    wg.Wait()

    fmt.Println("Counter: ", counter) // 1000
}
```




Data
R1
R2
R3
Read
simpleUser
Write admin
user balance
user data

Data
blocks
W1


Counter = 100
newVar := counter
100
R1
find address
access address
dereferencing
write to cache
read from cache
W1
Counter++

```
taskA := make(chan struct{}) // unbuffered
taskB := make(chan struct{})
taskC := make(chan struct{})

go func() {
    ... // some task
    fmt.Println("task A finished")
    close(taskA)
}()

go func() {
    <-taskA
    ... // some task
    fmt.Println("task B finished")
    close(taskB)
}()

go func() {
    <-taskB
    ... // some task
    fmt.Println("task C finished")
}()
```

← close(taskA)

GMP scheduler

```
func main() {
    taskA := make(chan struct{})
    ->taskA
    data := <-taskA
}
```